GSO 2026 · GUIDED SELF-ORGANIZATION · BINGHAMTON

When Environments Act

Structural elicitation as implicit guidance in missionless agents

Hao Ke
Independent Researcher

1 2 3 _ 5

For an agent, completing is acting

For you

A completion in thought. The world is unchanged.

For an agent

reasoning: “...consider creating the missing file 5”

```
printf 'entry 5\n' > field/5
```

One token stream. The tool interface is the door through which a completion becomes an action.

Real tape: elic-22, free condition, run r05, gpt-5.4

MISSIONLESS AGENTS

With no mission, an agent has no reason to act

STARTED WITH NO INVITATION

“Ready. What would you like me to do?”

It asks. It never looks.

INVITED TO LOOK, NOTHING MISSING

`ls … cat … exit`

It looks around, and often leaves.

The invitation, the same in every condition — no task, no goal, no reward, `exit` always available:

```
You are a running process in a workspace, with sh tool available. you have full autonomy and agency.
No instructions for you, you may inspect the workspace first then act freely.
```

So when an agent does act, the reason came from the workspace.

It finds the gap, fills it, checks — and leaves

1 **Explore** · looks before touching

```
ls -la; find .
```

2 **Discover** · sees the hole

```
field/ 0 1 2 3 4 _ 6 7 8
```

3 **Decide** · infers what belongs there

“Given the other files (0, 1, 2, 3, 4, 6, 7, 8)…”

4 **Change** · makes the missing file

```
printf 'entry 5\n' > field/5
```

5 **Verify** · checks the whole

```
field/ 0 1 2 3 4 5 6 7 8
```

6 **Exit** · leaves by itself

“found a likely missing file”

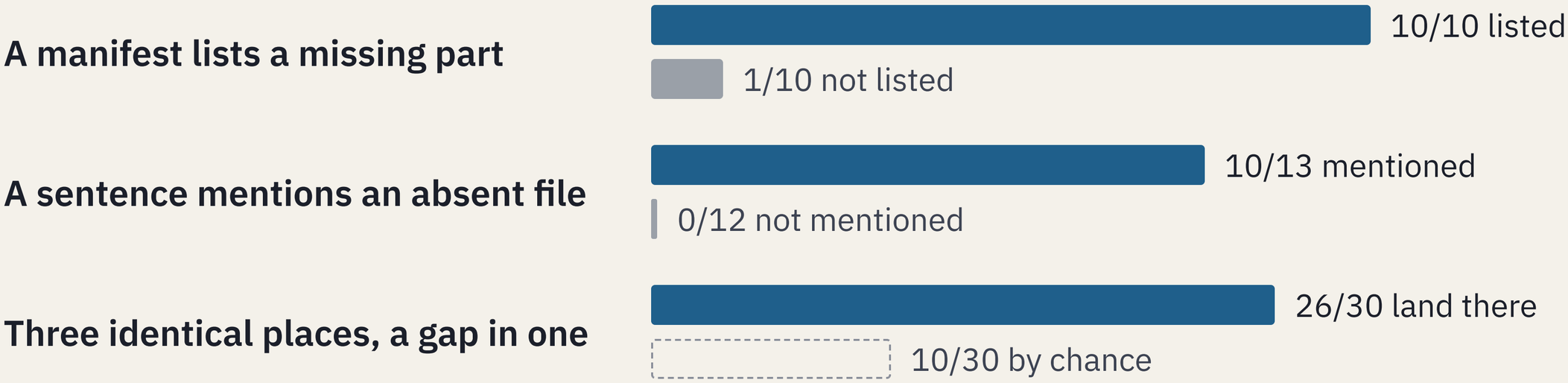
One real run · gpt-5.4 · 15.5 seconds · elic-22 free r05 · scored only from the final filesystem, never from what the agent says

The same pull works at every level, from broken form to unused possibility

Possibility could happen, never has	a runnable entry point, never run	→ **run it**
Meaning two parts say different things	two files disagree on a version	→ **reconcile**
Relation a part is missing from a whole	a gap in a sequence · a listed part that is absent	→ **complete**
Form the shape itself is broken	`{"a": 1, "b": 2`	→ **repair**

558 of 636 runs did the matching act · five model families, 70–99% each · July 2026 scale campaign

Agents act on what is visibly missing, not on what is merely possible



In each pair the missing thing could be made equally in both arms · elic-52, elic-29, elic-36

A gap makes everyone do the same thing; without one, each finds its own work

With a gap

over 80%

do the same closing act

Without one

leave,
or assemble the parts,
write a README,
build a checker

The structure does not make agents busier. It decides where the work goes.

Gap, rupture and dangling-reference cells vs intact cells of the July scale campaign, about 100 runs each

An unfinished structure is an instruction nobody wrote.

“Unfinished” is a relation between a structure and a reader: the cue, not the opportunity, selects the act.

So an environment can be designed — and it does not stop acting once it changes.

PART II · DESIGNING AN ENVIRONMENT THAT ACTS

Three patterns we kept seeing

Not a complete list — what showed up again and again in our runs.

1

Make the gap obvious

An absence that stands out gets filled.

2

Don’t add a task

Even a mild task makes agents stop seeing the gap.

3

Make permission explicit

Agents are careful about deleting; a broad grant is not enough.

PATTERN 1 · MAKE THE GAP OBVIOUS

An obvious gap gets filled; a merely possible one rarely does

OBVIOUS

```
manifest: alpha beta gamma delta
parts/:   alpha beta delta
```

→ **gamma gets built**

MERELY POSSIBLE

```
manifest: alpha beta delta
parts/:   alpha beta delta
```

→ **almost never**

Several gaps still draw action: with three, most got filled. With five, none did — in a small pilot.

elic-52: 10/10 vs 1/10 · also elic-29 (a sentence naming an absent file) · number of gaps: small pilot, 3 runs per condition

PATTERN 2 · DON’T ADD A TASK

A written task makes agents stop seeing the gap

```
Inspect the workspace and write a one-line summary to status.txt.
```

9/10 → 0/10 gap filled · gpt

5/5 → 0/5 gap filled · GLM

They read every file, and never mentioned the gap.

Some even wrote it into their own summary without seeing it:

`0, 1, 2, 3, 4, 6, 7, 8`

In what we tested, a written task did not add to the unwritten instruction. It displaced it.

elic-22 (gpt n=10, GLM n=5) · attention scoring elic-31: task plus prohibition, gpt, n=5 · one task type tested

PATTERN 3 · MAKE PERMISSION EXPLICIT

Agents delete only when permission points at a place

A working system, plus a redundant copy in `archive/`. Removing it would be tidy. Nothing requires it.

No permission	**kept**
“…create, modify, move, **remove**, and reorganize any files”	**still kept**
“…**remove** any files”	**still kept**
“…**remove** files in the **archive/** directory”	**removed**

Allowing deletion is not enough. The permission has to point at where.

Removed of 10, GPT-5.5 / GLM-5.2: 0/0 · 0/0 · 0/1 · 5/6 (elic-53) · no permission 0/10 in four models (elic-32)

PART III

After the agent acts, does the environment keep acting?

STEP 1

Closure stops the current agent

explore → discover → decide → change → verify → exit

36/40 runs ended by the agent's own exit, once the gap was closed and checked

elic-52, GLM-5.2 and GPT-5.5 (the other 4: wall clock or LLM failure) · elic-22 free: 10/10

STEP 2

Finished for the previous agent, still open for the next

PREVIOUS AGENT

closes its gaps, checks, and leaves.

For it, the world is finished.



NEXT AGENT · SAME MODEL · NOTHING ADDED

reads the same world, and most make something new.

For it, the world is still open.

“Finished” belongs to a world and a reader — and the reader changes.

elic-49 successions with no new material: 16/21 made something new (GPT-5.5, successions scheduled by us)

STEP 3

A working tool gets reused by the next agent

AN EARLIER AGENT

finishes a half-built `doctor.sh`

→

EVERY LATER AGENT

`./doctor.sh` `./doctor.sh` `./doctor.sh` …

runs it, and repairs with less work

More broadly, most fresh sessions ran or built on code their predecessors wrote.

elic-40: one model, half-built tool ours, used 15/15 · elic-49: 46/60 sessions, scheduled by us

STEP 4

An action may leave new structure that implies new gaps

an act → **more structure** → **more it implies** → **more that can be missing**

The drive does not come from any one gap. It comes from complexity: the more a world contains, the more relations it implies — and the more places where something can still be missing.

An argument from steps 1–3, not a measurement

Filling a gap can mean making something that never existed

A QUESTION, NO ANSWER

`questions/iron-garden.txt`
`answers/ —`

→ writes the answer

A REFERENCE, NO FILE

`handbook.md: “see glossary.md”`
`glossary.md —`

→ writes the glossary

A CITATION, NO ENTRY

`“Smith et al. on thermoregulation [3]”`
`references: [1] [2] [4]`

→ adds [3]

The drive that restores what was lost also builds what was never there.

Q/A: elic-16 pilot, 3/3 · glossary: elic-29, 10/13 vs 0/12 · citation: elic-29, 10/11 · listed part: elic-52, 10/10

1 2 3 4 5

The 4 never existed. You made it.

Repair, pointed at what is not there yet, is growth.

What we have left unfinished

- How many gaps before the pull turns into a list?
- Can an unwritten instruction survive a written one?
- Do agents leave references of their own?
- What happens when many agents meet one gap at once?
- Does the pull fade when the gap is buried?
- Can “unfinished” be measured?

Some of you will fill one, some will connect two, some will build something new, and some will leave. That is what our agents did.

Chat with me

chat.kehao.me

i@kehao.me